

A Technical Introduction to Rectifier neural networks

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

Section 1

Parametric ReLU (PReLU, 2016) takes this idea further by making α a learnable parameter along with the other network parameters. Note that for $\alpha \leq 1$, this is equivalent to

Section 2

where $b \geq 0$ is a hyperparameter that determines the "size" of the curved region near $x = 0$. (For example, letting $b = 0$ yields ReLU, and letting $b = 4$ yields the metallic mean function.) Squareplus shares many properties with softplus: It is monotonic, strictly positive, approaches 0 as $x \rightarrow -\infty$, approaches the identity as $x \rightarrow +\infty$, and is C^∞ smooth. However, squareplus can be computed using only algebraic functions, making it well-suited for settings where computational resources or instruction sets are limited. Additionally, squareplus requires no special consideration to ensure numerical stability when x is large.

Section 3

Non-differentiability at zero (however, it is differentiable anywhere else, and the value of the derivative at zero can be chosen to be 0 or 1 arbitrarily). Not zero-centered: ReLU outputs are always non-negative. This can make it harder for the network to learn during backpropagation, because gradient updates tend to push weights in one direction (positive or negative). Batch normalization can help address this. ReLU is unbounded. Redundancy of the parametrization: Because ReLU is scale-invariant, the network computes the exact same function by scaling the weights and biases in front of a ReLU activation by k , and the weights after by $1/k$. Dying ReLU: ReLU neurons can sometimes be pushed into states in which they become inactive for essentially all inputs. In this state, no gradients flow backward through the neuron, and so the neuron becomes stuck in a perpetually inactive state (it "dies"). This is a form of the vanishing gradient problem. In some cases, large numbers of neurons in a network can become stuck in dead states, effectively decreasing the model capacity and potentially even halting the learning process. This problem typically arises when the learning rate is set

too high. It may be mitigated by using "leaky" ReLU instead, where a small positive slope is assigned for $x < 0$. However, depending on the task, performance may be reduced.

Section 4

$$\ln(1 + e^x) \approx \begin{cases} \ln 2, & x = 0 \\ x - e^{-x} / \ln 2, & x \neq 0 \end{cases}$$

Section 5

DELU Extended Exponential Linear Unit (DELU, 2023) is an activation function which is smoother within the neighborhood of zero and sharper for bigger values, allowing better allocation of neurons in the learning process for higher performance. Thanks to its unique design, it has been shown that DELU may obtain higher classification accuracy than ReLU and ELU.

Section 6

In these formulas, α is a hyperparameter to be tuned with the constraint $\alpha \geq 0$. Given the same interpretation of α , ELU can be viewed as a smoothed version of a shifted ReLU (SReLU), which has the form $f(x) = \max(-\alpha, x)$.

Section 7

where x is the input to a neuron. This is analogous to half-wave rectification in electrical engineering. ReLU is one of the most popular activation functions for artificial neural networks, and finds application in computer vision and speech recognition using deep neural nets and computational neuroscience.

Section 8

ELU Exponential linear units (2015) smoothly allow negative values. This is an attempt to make the mean activations closer to zero, which speeds up learning. It has been shown that ELUs can obtain higher classification accuracy than ReLUs.

Section 9

where $\Phi(x) = P(X \leq x)$ is the cumulative distribution function of the standard normal distribution and $\operatorname{erf}(z)$ is the error function. This activation function is illustrated in the figure at the start of this article. It has a "bump" with negative derivative to the left of $x < 0$. It serves as the default activation for many transformer models such as BERT. The same paper also introduces a few faster approximations for GELU. The first approximation follows from an approximation for $\Phi(x)$

$\Phi(x)$:

Section 10

$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$, $f'(x) = \begin{cases} 1 & x > 0 \\ \alpha & x \leq 0 \end{cases}$

Section 11

$\text{LSE}(x_1, \dots, x_n) = \ln(e^{x_1} + \dots + e^{x_n})$

Section 12

The second, less-precise approximation uses the sigmoid (logistic) function as $f(x) \approx x \cdot \text{sigmoid}(1.702x)$, whose formula resembles SiLU (see below).

Section 13

In the context of artificial neural networks, the rectifier or ReLU (rectified linear unit) activation function is an activation function defined as the non-negative part of its argument, i.e., the ramp function:

Section 14

$\text{LSE}_0(x_1, \dots, x_n) := \text{LSE}(0, x_1, \dots, x_n) = \ln(1 + e^{x_1} + \dots + e^{x_n})$

Section 15

Sparse activation: for example, in a randomly initialized network, only about 50% of hidden units are activated (i.e. have a non-zero output). Better gradient propagation: fewer vanishing gradient problems compared to sigmoidal activation functions that saturate in both directions. Efficiency: only requires comparison and addition. Scale-invariant (homogeneous, or "intensity equivariance"):